

The Topology Engine + UI

Drawing the map, hanging it on the wall

What this chapter is about

By now we have three different "witnesses" who saw the network: the firewall ingestor, the SNMP crawler, and the active scanner. Each has put their notes into the database as `observation` rows. Some of these notes overlap ("I saw 10.0.5.42 too"). Some conflict ("I think it runs Linux"; "I think it runs Windows"). Some are stale ("I haven't seen 10.0.5.42 in 8 hours").

This chapter is about turning that pile of observations into **one coherent picture**: a graph of devices and links the customer can *look at* on a screen, click around, search, and see "what changed since last week."

Real-life analogy. *Three different surveyors walked through your factory and brought back rough notebooks. Surveyor A counted machines, Surveyor B traced the cabling, Surveyor C noted serial numbers off a sticker. Some pages agree, some don't, some are weeks old. The cartographer's job — that's us in this chapter — is to lay all three notebooks side by side, decide whose word to trust on what, draw a single clean map, mark which machines are currently humming and which are silent, and pin the map on the wall in the supervisor's office. When a surveyor brings a new notebook tomorrow, we don't throw the old map away — we save it as "yesterday's snapshot" and overlay the diff in red.*

Words you'll see (and what they mean)

Observation. A single fact tied to "when I saw it" and "who saw it." Built in Chapter 0. Examples: "MAC aa:bb:.. observed on port Gi1/0/14 of switch-A at 14:03 by collector-7," "IP 10.0.5.42 answered TCP/22 with banner SSH-2.0-OpenSSH_8.4 at 14:22 by collector-9."

Provenance. "Where did this fact come from?" Every entity in our database can answer it. If two sources disagree, provenance lets us pick the more trustworthy one and explain the choice.

Fact fusion. The process of taking many observations of the same logical thing (a device, an IP binding, an open port) and producing one current "best" answer.

Topology graph. A graph data structure where **nodes** are devices (and optionally subnets, sites) and **edges** are connections between them — LLDP/CDP cables, IPSec tunnels, L3 adjacencies, host-to-port placements.

Snapshot. A point-in-time copy of the topology graph. We keep these so the customer can ask "what did our network look like last Tuesday?" or "what changed in the last hour?".

Diff. The change between two snapshots. "These three devices are new, this device disappeared, this neighbour edge moved from port 14 to port 22."

Lifecycle state. Independent of "is it on right now?" — this answers "does this device still exist?". Three values: `active` (we believe it's part of the network), `decommissioned` (we haven't seen it in a while; flagged for review), `archived` (long absent; not shown on map by default but kept for history).

Availability state. The "is it on right now?" axis. Four values: `up`, `down`, `unknown`, `maintenance`. Independent from lifecycle — a decommissioned device can briefly appear "up" if someone plugs it in by mistake.

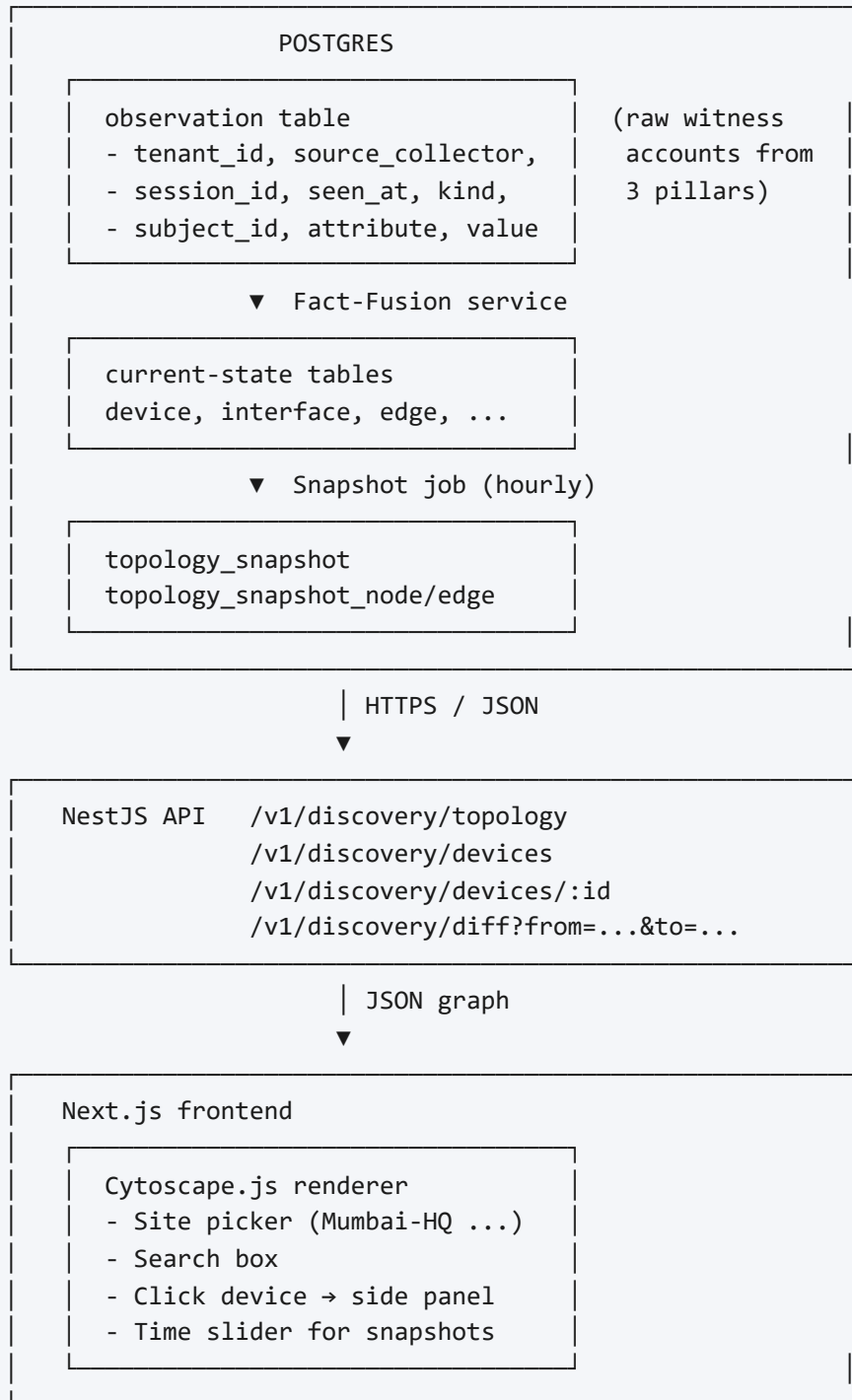
Hysteresis. The "wait before flapping" rule. We don't mark a device `down` after one missed ping; we wait for 3 consecutive failures. Same in reverse: 2 consecutive successes before we trust `up`. Prevents alert spam from one packet loss.

Cytoscape.js. A widely used JavaScript library for rendering interactive network graphs in the browser. Hands-down the standard choice for enterprise topology visualisation. Apache 2.0 licence.

Force-directed layout. A graph-drawing algorithm that pretends the edges are springs and the nodes repel each other; it lets the graph settle into a readable shape automatically. Default rendering style for topology maps.

NDJSON (Newline-Delimited JSON). One JSON object per line, no top-level array. Streamable. We use it on the WebSocket from collector to server so we can flush observations as we discover them.

What we're building (the picture)



The fact-fusion problem (and how we solve it)

Suppose we have three observations of the same logical device:

Witness	Says	Time
SNMP crawler	"chassis serial = FDO2412ZoAB, hostname = swA, vendor = Cisco"	14:00
Active scanner	"10.0.0.5 answered SNMP, sysObjectID points to Cisco IOS-XE, banner from SSH says swA"	14:22
Firewall ARP	"10.0.0.5 has MAC aa:bb:cc:dd:ee:01"	13:58

These are three accounts of one device. Fusion rules:

- Identity merge.** Match by chassis serial first; then by MAC; then by IP. The result is one logical `device` row with multiple aliases.
- Attribute precedence.** When two witnesses disagree on an attribute, prefer the one with stronger source. Ranking: SNMP (most authoritative) > firewall ingest > banner grab > ICMP-only. Per-attribute rules — vendor from SNMP wins, hostname from TLS-cert SAN can override DNS PTR, etc.
- Recency window.** Within the same source rank, latest `seen_at` wins.
- Conflict log.** When fusion changes its mind, write a row to `fusion_conflict` with both candidate values and the rule that resolved it. Audit & debug both depend on this.

Two-axis device state (the trick most homegrown systems get wrong)

The single biggest mistake when building one of these: conflating "is it powered on" with "does it still exist". They're independent.

	up	down	unknown	maintenance
active	Normal device, currently reachable	Active device, currently unreachable — alert	Just discovered or just lost contact	Operator marked it under planned work
decommissioned	Reappeared after long absence — investigate	Long absent; expected	Long absent; expected	n/a
archived	Hidden from default view but	Hidden from default view but	—	n/a

	retained for history	retained for history		
--	-------------------------	-------------------------	--	--

Rules of thumb:

- `active` → `decommissioned` automatic after configurable absence (default 7 days).
- `decommissioned` → `archived` automatic after a longer period (default 90 days).
- Either can be set manually by an operator at any time.
- Availability has hysteresis — 3 consecutive misses to flip `up`→`down` ; 2 consecutive hits to flip `down`→`up` .

Step-by-step plan

Step 1 — Build the fusion service

Create `backend/BE/src/discovery/fusion/fusion.service.ts` . Public API:

```
fuseSession(sessionId): Promise<FusionSummary>;
fuseTenant(tenantId, since: Date): Promise<FusionSummary>;
```

Triggers:

1. After every `ScanJobDone` from a collector → call `fuseSession` .
2. Hourly cron → call `fuseTenant` with the last hour, to clean up cross-session merges.

Internally: read observations newer than the last fuse-pointer, group by candidate identity (chassis ID → MAC → IP), upsert into current-state tables.

Step 2 — Build the snapshot service

Create `backend/BE/src/discovery/snapshot/snapshot.service.ts` . Hourly cron per tenant:

```
tx.begin();
INSERT INTO topology_snapshot (tenant_id, built_at, ...) RETURNING id AS sid;
INSERT INTO topology_snapshot_node SELECT sid, ... FROM device WHERE tenant_id=...;
INSERT INTO topology_snapshot_edge SELECT sid, ... FROM neighbor_edge WHERE ...;
tx.commit();
```

Retention: keep one snapshot per hour for 7 days; one per day for 90 days; one per month forever (rolling-window archival).

Step 3 — Build the diff endpoint

`GET /v1/discovery/diff?from={snapshotId}&to={snapshotId}` → `{nodesAdded, nodesRemoved, nodesChanged, edgesAdded, edgesRemoved, edgesChanged}` .

Implementation is pure SQL — set differences on the snapshot tables. No graph-library needed.

Step 4 — Build the topology read API

Endpoint	Returns
<code>GET /v1/discovery/sites</code>	List of sites for the current tenant.
<code>GET /v1/discovery/topology?siteId={id}&at={ts}</code>	Cytoscape-formatted JSON: <code>{nodes:[...], edges:[...]}</code> . <code>at</code> defaults to "now"; if specified, returns the snapshot closest to that time.
<code>GET /v1/discovery/devices?siteId={id}&q={search}</code>	Searchable list, paged.
<code>GET /v1/discovery/devices/:id</code>	Full detail: interfaces, IPs, MACs, open ports, recent observations, lifecycle & availability history.
<code>GET /v1/discovery/diff?from={sid}&to={sid}</code>	Snapshot diff.

Step 5 — Build the frontend page

1. Add `frontend/src/app/discovery/topology/page.tsx` in the existing Next.js app.
2. Install `cytoscape` and `cytoscape-fcose` (better force-directed layout).
3. Layout: left sidebar = site picker + search, centre = the graph, right sidebar = selected-device detail panel, top bar = time slider for snapshots.
4. Node styling by device kind: router, switch, firewall, server, printer, IoT, unknown — distinct icons. Availability shown as a coloured dot (green/red/grey/blue).
5. Edge styling: solid for L2 LLDP/CDP, dashed for L3 routing adjacency, double-dashed for IPSec tunnels.

Step 6 — Add change-event surfacing

When the snapshot diff produces an interesting change, write a `discovery_event` row:

- `device_added`

- `device_disappeared`
- `device_moved_port` (same MAC, different switch port)
- `new_open_port`
- `new_neighbor_edge`

Surface these on the dashboard as a feed. Chapter 5 turns them into alerts.

Step 7 — Time-slider snapshots

The frontend's time slider lets the user drag back through hourly snapshots. Loads via `?at=` on the topology endpoint. The graph re-renders, and changed entities animate (added = fade-in green, removed = fade-out red, moved = animated edge swap).

How we test Chapter 4

Test 1 — Identity merge.

1. Inject observations from all 3 pillars for the same device (firewall ARP + SNMP system + active SSH banner).
2. Confirm exactly ONE row in `device`, with all three IPs/MACs aliased to it.
3. Confirm a `fusion_conflict` row if vendor names disagreed, resolved per precedence.

Test 2 — Snapshot & diff.

1. Trigger snapshot at T₀.
2. Add a new device, change a banner, remove an old device.
3. Trigger snapshot at T₁.
4. Diff endpoint returns exactly: 1 added, 1 removed, 1 changed.

Test 3 — Lifecycle automation.

1. Mark a device's `last_seen_at` as 8 days ago.
2. Run lifecycle cron.
3. Device flips to `decommissioned`; an event row is written.

Test 4 — Availability hysteresis.

1. Simulate one missed ping → device stays `up` .
2. Three in a row → device flips to `down` .
3. Two successful pings → device flips back to `up` .

Test 5 — Frontend rendering.

1. Build a 50-device synthetic graph.
2. Page loads in < 2 s; graph stabilises in < 3 s.
3. Click on a device → side panel shows interfaces, IPs, MACs, recent observations.
4. Search "swA" → graph centres on that device.
5. Time slider 1 h back → graph re-renders to that snapshot.

Test 6 — Cross-tenant isolation in API.

1. Auth as tenant Acme.
2. Hit `GET /v1/discovery/devices/:id` with an ID that belongs to Globex.
3. Expect HTTP 404, not 403, not 200, never the actual record. (RLS from Chapter 0 should make this trivial.)

Things that can go wrong (and what to watch for)

Identity merge collisions. Two physically different devices share the same MAC because some network card driver reuses a default. Always use chassis serial as primary key when available; only fall back to MAC when serial is absent. Log when MAC was used as the merge key.

Stale current-state. If the fusion service crashes mid-run, current-state can become inconsistent. Make fusion idempotent and re-runnable for a session ID. Always wrap the upsert in a transaction.

Snapshot bloat. Storing every-hour snapshots forever explodes the table. Implement the rolling-window retention (1h × 7d, 1d × 90d, 1m × forever) from day one — it's much harder to add later.

Cytoscape rendering 5,000 nodes. Above ~1,500 nodes, Cytoscape's default force-directed layout becomes sluggish. Switch to `fcose` or `cose-bilkent` ; group "host clouds" behind a switch into a single collapsed node that expands on click.

Time-slider race conditions. User drags slider quickly; we fire 30 requests; they arrive out of order. Always track the last requested timestamp and discard responses that aren't for the latest.

Conflicting observations creating flapping rows. Two collectors running at once can both update the same device with slightly different attributes. Use Postgres advisory locks keyed by `device_id` during fusion to serialise updates.

Lifecycle automation deleting real data. Never automatically delete; only mark `archived` . Deletion is operator-initiated and must require confirmation.

Showing wrong snapshot in the UI. If the API returns "snapshot closest to `at` ", be explicit about whether that's before or after; show the actual `built_at` timestamp prominently in the UI so the user knows what they're looking at.

Exit checklist (you can move to Chapter 5 only when ALL are true)

- ☐ Fact-fusion produces one device row when fed three witnesses for the same logical device.
- ☐ Snapshot+diff endpoints work and return correct add/remove/change counts on a known scenario.
- ☐ Lifecycle (active/decommissioned/archived) and availability (up/down/unknown/maintenance) are independent and tested.
- ☐ Hysteresis verified: 1 miss does not flip state, 3 misses does.
- ☐ Frontend renders a 1,500-node tenant graph in < 5 s after layout.
- ☐ Search, click-to-detail, and time-slider all work.
- ☐ Cross-tenant API access is impossible (returns 404 on out-of-tenant IDs).
- ☐ At least 5 distinct event types (device added/disappeared/moved/new-port/new-edge) are recorded by the diff service.

Recap card — Chapter 4 in 5 sentences

1. The three pillars dump observations into one table; fact-fusion turns them into one current-state row per logical device, with provenance and conflict logs.
2. Hourly snapshots of the topology graph are stored on a rolling-window retention so the customer can travel back in time and diff.
3. Every device has two independent state axes — lifecycle (active/decommissioned/archived) and availability (up/down/unknown/maintenance) — and conflating them is the most common bug to avoid.
4. The frontend renders the graph with Cytoscape.js (force-directed layout), with site picker, search, click-to-detail, and a time slider.
5. Diff between snapshots produces actionable events (device added, disappeared, moved port, new open port) — these become alerts in Chapter 5.